Derangadic: A Combinatorial Number System for Derangements

with Amortized $O(1)$ Stateful Lexicographic Generation

Deepesh Patel Aditya Patel

JNumberTools – Open-Source Combinatorics Library

https://github.com/deepeshpatel/jnumbertools

May 2026

Abstract

We introduce the *Derangadic Number System*, a variable-length, mixed-radix positional representation that establishes a canonical bijection between the non-negative integers $\{0, 1, \ldots, !n - 1\}$ and the lexicographically ordered derangements of n elements, where $!n$ denotes the subfactorial. Unlike the classical factorial number system (Factoradic) or its generalisations for permutations (Permutadic), the Derangadic encoding respects the fixed-point-free constraint intrinsically: every valid digit sequence maps to a derangement and to nothing else.

A key structural property—*parity-locked stabilisation*—shows that the encoding of any rank r depends only on r and the parity of n, not on n itself once n is large enough. This decouples the active encoding width from the global universe size.

We formalise four core properties (existence, uniqueness, bijection, order preservation) with complete proofs, prove the critical invariant that the least-significant digit is always zero, and handle dead-end avoidance when exactly two positions remain. We derive a deterministic stateful successor machine (`DerangadicIncrement`) that advances through all derangements in lexicographic order. We prove that its per-step cost is amortised $O(1)$: the expected carry length $\sum_{k=1}^{\infty} k/!k$ converges to a finite constant (bounded by e^2) because $k/!k \sim ek/k! \to 0$ super-exponentially.

We further establish that the Factoradic number system is the unrestricted special case of Derangadic obtained by setting the restricted count $c_s = 0$ at all encoding steps, collapsing restricted derangement counts to factorials. This places Derangadic in a unified combinatorial hierarchy alongside Permutadic, with Factoradic as their common limit. Empirical benchmarks on an Apple MacBook Air (M5, 16 GB) using the JNumberTools Java library confirm the theory: the stateful increment operates at a flat $\approx$ 75–85 ns per step for n ranging from 100 to 50,000.

Contents

1 Introduction

Positional number systems assign meaning to sequences of digits by associating each position with a weight. The decimal system uses powers of ten; the Factoradic [4] uses factorial weights and provides a canonical bijection with permutations via the Lehmer code. This idea has been extended in various directions: the Combinadic (binomial number system) encodes combinations [3], and the Permutadic system of Patel and Patel encodes k-permutations [6].

A *derangement* is a permutation with no fixed point, i.e., $\pi(i) \neq i$ for all positions i. The study of derangements is classical—they are counted by the subfactorial $!n$, satisfy the inclusion-exclusion formula, and arise in problems ranging from hat checks to Latin squares. Despite this classical status, the problem of assigning a *canonical rank* to each derangement in lexicographic order—and recovering a derangement from its rank efficiently—has received comparatively little attention. Mikawa and Taura [5] gave ranking/unranking algorithms; Korsh and LaFollette [2] gave constant-time generation but in a non-lexicographic order.

The contribution of this paper is threefold.

1. We introduce the *Derangadic Number System*: a rigorous mixed-radix representation whose digit sequences are in bijection with lexicographically ordered derangements.

2. We prove four fundamental properties with full mathematical rigour, establish the *parity-locked stabilisation* theorem, prove the *LSD invariant* ($D_0 = 0$ always), and formalize dead-end avoidance.

3. We derive a deterministic stateful successor machine and prove its amortised $O(1)$ complexity via convergence of the expected carry length series. We also prove that Factoradic emerges as the unrestricted limit of Derangadic, unifying it with Permutadic under a common combinatorial framework.

Paper organisation. Section 2 collects mathematical preliminaries. Section 3 defines the Derangadic system and establishes its fundamental properties. Section 4 proves parity-locked stabilisation. Section 5 gives ranking, unranking, and the increment machine. Section 6 contains the amortised complexity proof. Section 7 establishes the theoretical connection to Factoradic and Permutadic. Section 9 reports benchmarks with visualizations. Section 10 discusses applications and generalizations.

Notation and Conventions

Throughout the paper, $[n]$ denotes the set $\{0, 1, \ldots, n-1\}$.

Digit ordering convention. Derangadic representations are displayed **MSD-first** (most-significant digit first):

$$[D_{k-1},\ D_{k-2},\ \ldots,\ D_1,\ D_0],$$

where D_{k-1} is the most significant digit and D_0 is the least significant digit. This matches the natural reading order of numbers.

Implementation note. The underlying implementation (JNumberTools) stores digits in the *opposite* order: `digits[0]`$= D_0$ (LSD) and `digits[k-1]`$= D_{k-1}$ (MSD). This implementation detail is noted wherever it matters for the algorithmic description.

Trailing zeros. The implementation preserves trailing zeros to maintain consistent array length. When comparing encodings, trailing zeros beyond the last non-zero digit may be ignored as they do not affect the rank value.

2 Mathematical Preliminaries

2.1 Derangements and the Subfactorial

Definition 2.1 (Derangement)**.** *A* derangement *of* $[n]$ *is a bijection* $\pi : [n] \to [n]$ *such that* $\pi(i) \neq i$ *for every* $i \in [n]$*. The set of all derangements of* $[n]$ *is denoted* $\mathcal{D}_n$*.*

Definition 2.2 (Subfactorial)**.** *The* subfactorial $!n = \|\mathcal{D}_n\|$ *satisfies*

$$!n = n! \sum_{k=0}^{n} \frac{(-1)^k}{k!} = \left\lfloor \frac{n!}{e} + \frac{1}{2} \right\rfloor, \qquad n \geq 1, \tag{1}$$

with $!0 = 1$, $!1 = 0$*, and the recurrence*

$$!n = (n-1)\big(!n-1 + !n-2\big), \qquad n \geq 2. \tag{2}$$

Lemma 2.1 (Parity and Monotonicity of Subfactorials)**.** $!n = 0$ *iff* $n = 1$*. For all* $n \geq 2$, $!n \geq 1$*, and the subfactorial sequence is strictly increasing on each parity class:* $!n > !n-2$ *for* $n \geq 4$*.*

Proof. Follows immediately from recurrence (2). For $n \geq 2$, $(n-1) \geq 1$ and $!n-1 + !n-2 \geq 1$, so $!n \geq 1$. The strict inequality $!n > !n-2$ follows from $!n = (n-1)(!n-1 + !n-2) \geq 3 \cdot !n-2 > !n-2$ for $n \geq 4$. □

2.2 Restricted Derangements

Definition 2.3 (Restricted Derangements)**.** *Let* $m, r \in \mathbb{N}$ *with* $0 \leq r \leq m$*.* $\mathrm{RD}(m, r)$ *denotes the number of permutations of* m *elements in which exactly* r *specified positions are* forbidden *from containing their natural value. Equivalently,* $\mathrm{RD}(m, r)$ *is the number of permutations of* $\{0, \ldots, m-1\}$ *that avoid fixed points at a specified set of* r *positions.*

Lemma 2.2 (Closed Form of Restricted Derangements)**.**

$$\mathrm{RD}(m, r) = \sum_{k=0}^{r} (-1)^k \binom{r}{k} (m-k)! \tag{3}$$

Proof. Standard inclusion-exclusion: choose k of the r forbidden positions, force those k to be fixed (contributing $(m-k)!$ arrangements), and alternate signs to correct for overcounting. □

Corollary 2.3. $\mathrm{RD}(m, m) = !m$ *and* $\mathrm{RD}(m, 0) = m!$*.*

Proof. When all m positions are forbidden, we count derangements. When no positions are forbidden, we count all permutations. □

2.3 Lexicographic Order on Derangements

For a derangement $\pi \in \mathcal{D}_n$, we write π as the sequence $(\pi(0), \pi(1), \ldots, \pi(n-1))$. Lexicographic order on $\mathcal{D}_n$ is the restriction of the standard lexicographic order on sequences in $[n]^n$ to the subset $\mathcal{D}_n$.

Remark 2.1. $\mathcal{D}_n$ *inherits a total order from the lexicographic order on all permutations. The* rank *of a derangement is its 0-based position in this ordering.*

3 The Derangadic Number System

3.1 Motivating Idea

In the Factoradic, one writes the rank r of a permutation as $r = \sum_{i=0}^{n-1} d_i \cdot i!$ with $0 \le d_i \le i$. The digit d_i at step i counts how many available elements precede the chosen one, and the block sizes are factorials because all completions are unconstrained.

In the Derangadic, completions must be derangements, so the block sizes are restricted derangement counts. Moreover, the block size at each step depends on how many "forbidden" positions remain in the suffix—leading to a non-uniform, context-sensitive digit alphabet.

3.2 Active Length: actualN

Definition 3.1 (actualN)**.** *For $n \ge 2$ and rank r with $0 \le r < !n$, define*

$$\mathrm{actualN}(n, r) = \min\{k \in \mathbb{N} \mid k \equiv n \pmod 2,\ !k > r\}. \tag{4}$$

Remark 3.1. *Because $!n > r$ and $n \equiv n \pmod 2$, the set in (4) is non-empty and* $\mathrm{actualN}(n, r) \le n$*. The parity constraint ensures that the encoding respects the alternating structure of the subfactorial. Concretely, if n is even (resp. odd),* actualN *is always even (resp. odd).*

The quantity $\mathrm{actualN}(n, r)$ is the number of *active* digits in the encoding; the leading $n - \mathrm{actualN}$ positions of the full n-element derangement are determined greedily and do not carry any digit.

Algorithm 1 Computing $\mathrm{actualN}(n, r)$ (matching Java `smallestN`)

Require: $n \ge 2$, $0 \le r < !n$
Ensure: $k = \mathrm{actualN}(n, r)$

```
k ← n
while k > 2 and !k − 2 > r do
    k ← k − 2                                   ▷ Preserve parity
end while
return k
```

3.3 Digit Alphabet and Block Sizes

At encoding step s ($0 \le s < \mathrm{actualN}$), we have placed the first s elements of the active portion of the derangement and must choose the $(s+1)$-th. Let:

- $\mathrm{remaining} = \mathrm{actualN} - s$ be the number of positions yet to fill,
- c_s be the number of "restricted" positions among the remaining remaining positions (positions $j \ge s$ in the active block such that element j has not yet been used), i.e., those where a choice would create a fixed point.

If we pick element e at position s:

$$\begin{aligned} c_{s+1} &= c_s - 2 && \text{if both position } s \text{ and } e \text{ were restricted,} \\ c_{s+1} &= c_s - 1 && \text{if exactly one was restricted,} \\ c_{s+1} &= c_s && \text{if neither was restricted.} \end{aligned}$$

The number of valid completions after choosing e is $\mathrm{RD}(\mathrm{remaining} - 1, c_{s+1})$. The digit D_{k-1-s} (in MSD-first notation) records the 0-based rank of e among the legal candidates for position s, sorted in increasing order of element value.

Definition 3.2 (Digit Bounds)**.** *At step s (MSD-first index $i = k-1-s$, LSD-first index $j = s$):*

$$\text{maxDigit}(i) = \begin{cases} 0 & \text{if } remaining = 1, \\ (remaining - 1) - \mathbf{1}_{\{s \text{ not used}\}} & \text{otherwise}, \end{cases} \tag{5}$$

$$\text{minDigit}(i) = \min\{d \geq 0 \mid \text{choosing candidate } d \text{ does not lead to dead end}\}. \tag{6}$$

Remark 3.2 (Dead-end Detection)**.** *When $remaining = 2$, choosing a candidate that leaves the other element equal to its position creates an impossible completion (the last element would be forced to its own index). The* minDigit(i) *function skips such candidates.*

3.4 The LSD is Always Zero

Theorem 3.1 (LSD Invariant)**.** *For any valid Derangadic encoding $\mathbf{D}(r, n) = [D_{k-1}, \ldots, D_1, D_0]$ with $k = \text{actualN}(n, r) \geq 2$, the least-significant digit satisfies $D_0 = 0$.*

Proof. When $s = k - 1$ (the final step), exactly two positions remain: the current position $p = k - 1$ and one other position q. Exactly two elements remain: call them a and b. The derangement constraint requires:

- $\pi(p) \neq p$ and $\pi(q) \neq q$,
- π must be a bijection on $\{a, b\}$.

There are exactly two assignments: $(\pi(p), \pi(q)) = (a, b)$ or (b, a). At most one of these creates a fixed point (if $a = p$ or $b = q$). Since we are encoding a valid derangement, exactly one assignment is legal. Therefore, at step $s = k - 1$, there is exactly one legal candidate, so the digit D_0 (which indexes the chosen candidate among legal candidates) must be 0. □

Corollary 3.2 (Trailing Zero Structure)**.** *All valid Derangadic encodings end with at least one zero. Trailing zeros beyond the first may be padding for length consistency but do not affect the rank value.*

Remark 3.3 (Implementation Consequence)**.** *The Java implementation stores digits LSD-first:* `digits[0]` *is D_0. Since $D_0 = 0$ always,* `digits[0]` *is always* 0*. The increment machine starts scanning for a pivot at index* 1 *(the second-least-significant digit), as index* 0 *can never carry.*

3.5 Dead-End Avoidance

Lemma 3.3 (Dead-End Avoidance at remaining = 2)**.** *When $remaining = 2$, exactly two positions p, q and two elements a, b remain. Choosing candidate a for position p is illegal if and only if $b = q$. In that case, the candidate is skipped, effectively increasing* minDigit(i) *by 1 for that step.*

Proof. If $b = q$, placing a at p forces b to map to q, creating a fixed point. Since derangements cannot have fixed points, this path yields 0 completions ($\text{RD}(1, 1) = 0$). The implementation detects this case in `computeMinDigit` and increments the seen-candidate counter, skipping the dead-end candidate. □

3.6 Verified Examples

Example 3.1 (Even parity, $n = 12$, ranks 0–29)**.** *Table 1 lists the first 30 Derangadic encodings for $n = 12$ (even parity), verified against the JNumberTools Java implementation. All encodings end with $D_0 = 0$ as required by Theorem 3.1.*

Example 3.2 (Odd parity, $n = 13$, ranks 0–29)**.** *Table 2 lists the first 30 Derangadic encodings for $n = 13$ (odd parity), verified against the JNumberTools Java implementation. All encodings end with $D_0 = 0$.*

Table 1: Verified Derangadic encodings for $n = 12$ (even), ranks 0–29. Format: $[D_{k-1}, \ldots, D_1, D_0]$ (MSD-first). Note $D_0 = 0$ always.

Rank r	k	actualN	Derangadic (MSD-first)
0	2	2	$[0, 0]$
1	4	4	$[0, 1, 1, 0]$
2	4	4	$[0, 2, 0, 0]$
3	4	4	$[1, 0, 1, 0]$
4	4	4	$[1, 1, 0, 0]$
5	4	4	$[1, 1, 1, 0]$
6	4	4	$[2, 0, 0, 0]$
7	4	4	$[2, 1, 0, 0]$
8	4	4	$[2, 1, 1, 0]$
9	6	6	$[0, 1, 0, 0, 1, 0]$
10	6	6	$[0, 1, 0, 1, 0, 0]$
11	6	6	$[0, 1, 1, 0, 0, 0]$
12	6	6	$[0, 1, 1, 1, 1, 0]$
13	6	6	$[0, 1, 1, 2, 0, 0]$
14	6	6	$[0, 1, 2, 0, 1, 0]$
15	6	6	$[0, 1, 2, 1, 0, 0]$
16	6	6	$[0, 1, 2, 1, 1, 0]$
17	6	6	$[0, 1, 3, 0, 0, 0]$
18	6	6	$[0, 1, 3, 1, 0, 0]$
19	6	6	$[0, 1, 3, 1, 1, 0]$
20	6	6	$[0, 2, 0, 0, 0, 0]$
21	6	6	$[0, 2, 0, 1, 1, 0]$
22	6	6	$[0, 2, 0, 2, 0, 0]$
23	6	6	$[0, 2, 1, 0, 1, 0]$
24	6	6	$[0, 2, 1, 1, 1, 0]$
25	6	6	$[0, 2, 1, 2, 0, 0]$
26	6	6	$[0, 2, 1, 2, 1, 0]$
27	6	6	$[0, 2, 2, 0, 0, 0]$
28	6	6	$[0, 2, 2, 1, 0, 0]$
29	6	6	$[0, 2, 2, 2, 0, 0]$

3.7 Fundamental Properties

Theorem 3.4 (Existence)**.** *For every $n \geq 2$ and every r with $0 \leq r < !n$, the procedure in Definition ?? terminates and produces a well-defined digit tuple $\mathbf{D}(r, n)$.*

Proof. We show that at each step s, the running remainder r' satisfies $0 \leq r' < \mathrm{RD}(k - s, c_s)$ (the total count of valid completions from that step onward). This is true initially by definition of actualN. At step s, the legal candidates have block sizes $B_0, B_1, \ldots$ with $\sum_j B_j = \mathrm{RD}(k - s, c_s)$. Hence there exists a unique d with $\sum_{j<d} B_j \leq r' < \sum_{j \leq d} B_j$. After subtracting, the new remainder satisfies $0 \leq r' - \sum_{j<d} B_j < B_d = \mathrm{RD}(k - s - 1, c_{s+1})$, preserving the invariant. Termination follows since s increases by 1 at each step and $s < k$. □

Theorem 3.5 (Uniqueness)**.** *The digit tuple $\mathbf{D}(r, n)$ is unique: no two distinct ranks $r_1 \neq r_2$ in $[0, !n)$ yield the same digit sequence of the same length.*

Proof. Suppose $r_1 < r_2$. Let s^* be the first step where the two encoding procedures diverge. At step s^* the running remainders differ, so the unique d chosen differs. All subsequent digits may differ, but the tuple $\mathbf{D}(r_1, n) \neq \mathbf{D}(r_2, n)$ already at position $k - 1 - s^*$ (MSD-first). □

Table 2: Verified Derangadic encodings for $n = 13$ (odd), ranks 0–29. Format: $[D_{k-1}, \ldots, D_1, D_0]$ (MSD-first). Note $D_0 = 0$ always.

Rank r	k	actualN	Derangadic (MSD-first)
0	3	3	$[0, 1, 0]$
1	3	3	$[1, 0, 0]$
2	5	5	$[0, 1, 0, 0, 0]$
3	5	5	$[0, 1, 1, 1, 0]$
4	5	5	$[0, 1, 2, 0, 0]$
5	5	5	$[0, 2, 0, 1, 0]$
6	5	5	$[0, 2, 1, 0, 0]$
7	5	5	$[0, 2, 1, 1, 0]$
8	5	5	$[0, 3, 0, 0, 0]$
9	5	5	$[0, 3, 1, 0, 0]$
10	5	5	$[0, 3, 1, 1, 0]$
11	5	5	$[1, 0, 0, 0, 0]$
12	5	5	$[1, 0, 1, 1, 0]$
13	5	5	$[1, 0, 2, 0, 0]$
14	5	5	$[1, 1, 0, 1, 0]$
15	5	5	$[1, 1, 1, 1, 0]$
16	5	5	$[1, 1, 2, 0, 0]$
17	5	5	$[1, 1, 2, 1, 0]$
18	5	5	$[1, 2, 0, 0, 0]$
19	5	5	$[1, 2, 1, 0, 0]$
20	5	5	$[1, 2, 2, 0, 0]$
21	5	5	$[1, 2, 2, 1, 0]$
22	5	5	$[2, 0, 0, 1, 0]$
23	5	5	$[2, 0, 1, 0, 0]$
24	5	5	$[2, 0, 1, 1, 0]$
25	5	5	$[2, 1, 0, 1, 0]$
26	5	5	$[2, 1, 1, 1, 0]$
27	5	5	$[2, 1, 2, 0, 0]$
28	5	5	$[2, 1, 2, 1, 0]$
29	5	5	$[2, 2, 0, 0, 0]$

Theorem 3.6 (Bijection)**.** *The map $r \mapsto \mathbf{D}(r, n)$ is a bijection between $\{0, 1, \ldots, !n - 1\}$ and the set of valid Derangadic digit tuples of length $k = \text{actualN}(n, r)$. Equivalently, the composite map*

$$r \xmapsto{\text{toDerangadic}} \mathbf{D} \xmapsto{\text{toDerangement}} \pi$$

is a bijection between $\{0, \ldots, !n - 1\}$ and $\mathcal{D}_n$.

Proof. Injectivity: Theorem 3.5. Surjectivity: Every derangement $\pi \in \mathcal{D}_n$ can be encoded by reading off its digit at each step (the index of $\pi(s)$ among the legal candidates), giving a valid tuple; the inverse map fromDerangadic reconstructs r by summing the block-size offsets. The maps are mutual inverses by construction. □

Theorem 3.7 (Order Preservation)**.** *For $0 \leq r_1 < r_2 < !n$, the derangement $\pi_1 = \pi(r_1)$ is lexicographically smaller than $\pi_2 = \pi(r_2)$.*

Proof. At the first step s^* where the two encoding procedures diverge, rank r_1 selects a smaller digit $d_1 < d_2$, hence a smaller candidate element $e_1 < e_2$ for position s^*. Since earlier positions

$0, \ldots, s^* - 1$ chose identical elements, and at position s^* we have $e_1 < e_2$, lexicographic order gives $\pi_1 <_{\text{lex}} \pi_2$. □

4 Parity-Locked Stabilisation

A striking property of the Derangadic system is that the encoding of a rank r does not depend on n as long as n has the right parity and $!n > r$.

Theorem 4.1 (Parity-Locked Stabilisation)**.** *Let n_1, n_2 be two integers with $n_1 \equiv n_2 \pmod 2$, $n_1, n_2 \geq 2$, and let $r < \min(!n_1, !n_2)$. Then*

$$\text{actualN}(n_1, r) = \text{actualN}(n_2, r) =: k,$$

and the digit tuples $\mathbf{D}(r, n_1)$ and $\mathbf{D}(r, n_2)$ are identical.

Proof. By Definition 3.1, $k = \text{actualN}(n_i, r)$ is determined solely by r and the parity of n_i. Since $n_1 \equiv n_2 \pmod 2$, both give the same k. The digit computation in Definition **??** operates entirely on $[k] = \{0, \ldots, k-1\}$ and uses only block sizes $\text{RD}(k - s - 1, c_{s+1})$, which depend only on the local state of the encoding (step s, used-element flags within $[k]$). These quantities are identical for n_1 and n_2. Hence $\mathbf{D}(r, n_1) = \mathbf{D}(r, n_2)$. □

Corollary 4.2 (Size Independence)**.** *The Derangadic encoding of a rank r is independent of the universe size n, provided $n \equiv \text{actualN}(n, r) \pmod 2$ and $n \geq \text{actualN}(n, r)$. In particular, for large n the encoding uses only $\text{actualN} \ll n$ active digits.*

Corollary 4.3 (Compact Representation)**.** *A derangement of $n = 10^6$ elements at rank $r < !20 = 895{,}014{,}631{,}192{,}902{,}121$ is encoded by at most 20 digits, regardless of how large n is.*

Practical implication. This property means the increment machine (Section 5) never has to touch all n elements when performing a local carry; it operates only on the active window of $k = \text{actualN}$ digits. The global size n appears only when materialising the full derangement array.

5 Ranking, Unranking, and the Increment Machine

5.1 Ranking: Derangement to Rank

Definition 5.1 (fromDerangadic)**.** *Given a digit tuple $\mathbf{D} = [D_{k-1}, \ldots, D_0]$ of length k and order $n \geq k$, the rank is*

$$r = \sum_{s=0}^{k-1} \Big(\sum_{j=0}^{D_{k-1-s}-1} B_j^{(s)} \Big), \tag{7}$$

where $B_j^{(s)} = \text{RD}(k - s - 1, c_{s+1}^{(j)})$ are the block sizes at step s for candidate j.

Complexity. Each step computes $\text{RD}(\cdot, \cdot)$ via (3), which takes $O(r)$ operations for r restricted positions. Over k steps the total cost is $O(k^2)$.

5.2 Unranking: Rank to Derangement

Two implementations are provided in JNumberTools:

Array-based ($O(n^2)$, small n). After computing the digit tuple via Definition **??** (cost $O(k^2)$), iterate over positions and select the D_{k-1-s}-th legal element by scanning the available array.

Fenwick-tree-based ($O(n + k \log n)$, large n). A Fenwick (binary-indexed) tree [1] maintains the multiset of available elements. The d-th legal element at position s is found by an order-statistic query in $O(\log n)$, giving total cost $O(n + k \log n)$. Since $k \leq n$, this is $O(n \log n)$ in the worst case but typically $O(n)$ for the prefix fill plus $O(k \log n)$ for the active portion.

5.3 The Derangadic Increment Machine

The central data structure is a *state* σ comprising:

- $\mathbf{d} = [d_0, \ldots, d_{k-1}]$: current digits (LSD-first, as in the implementation);
- $\mathbf{M} = [M_0, \ldots, M_{k-1}]$: maximum digit at each position;
- $\boldsymbol{\pi}$: current derangement array of length n;
- a Fenwick tree of available elements;
- auxiliary per-step consumed-element and used-flag arrays.

The machine maintains a *dual-state invariant*: the encoded digits and the materialized derangement array are always consistent. Thus, `encodedToDerangement()` operates in $O(1)$ by simply returning the cached $\boldsymbol{\pi}$.

Definition 5.2 (Derangadic State Machine)**.** *The* Derangadic Increment Machine *is a deterministic transition system $\mathcal{M} = (\mathcal{S}, \Sigma, \delta, \sigma_0, \gamma)$ where $\mathcal{S}$ is the set of valid states, $\Sigma = \{\texttt{inc}\}$, $\delta : \mathcal{S} \to \mathcal{S}$ is the transition function, σ_0 is the initial state for rank r_0 and order n, and $\gamma : \mathcal{S} \to \mathcal{D}_n$ returns the current derangement.*

Transition function δ. On input `inc` at state σ with actualN $= k$:

1. **Find pivot.** Scan positions $i = 1, 2, \ldots, k-1$ (in the implementation's LSD-first order, so $i = 1$ corresponds to the second-least-significant digit). The pivot p is the smallest i with $d_i < M_i$.

   In MSD-first paper notation, the pivot is the *rightmost* non-maximal digit D_j with $D_j <$ maxDigit(j).

2. **Suffix increment.** If p exists:

   (a) Set $d_p \leftarrow d_p + 1$.

   (b) For all $i < p$: set $d_i \leftarrow$ minDigit(i) (reset less-significant digits).

   (c) **Rollback**: return to the Fenwick tree the elements placed at positions $s \geq k - 1 - p$ (the affected suffix).

   (d) **Rebuild**: re-fill those positions greedily from digit 0, using the Fenwick tree for $O(\log n)$ order-statistic queries.

3. **Structural expansion.** If no pivot exists ($k = n$, all digits maximal), the enumeration is complete. If $k < n$, set $k \leftarrow k + 2$, compute the first rank of the new k-layer as $r_{\text{new}} = !k - 2$ (the previous boundary), compute new digits via toDerangadic(r_{new}, n), and rebuild the entire active block.

Theorem 5.1 (Correctness of the Increment Machine)**.** *Starting from state σ_0 encoding rank r_0, successive applications of δ produce, in order, the derangements of ranks $r_0, r_0 + 1, \ldots, !n - 1$.*

Algorithm 2 Derangadic Increment (δ)

Require: State σ with digits $[d_0, \ldots, d_{k-1}]$ (LSD-first), max digits $[M_0, \ldots, M_{k-1}]$, Fenwick tree *avail*, derangement $\boldsymbol{\pi}$, offset $= n - k$.
Ensure: State updated to next rank; returns `true` if successful.

$p \leftarrow -1$
for $i = 1$ **to** $k - 1$ **do** ▷ Skip $i = 0$: LSD always 0, never carries
  if $d_i < M_i$ **then**
    $p \leftarrow i$; **break**
  end if
end for
if $p \neq -1$ **then** ▷ Suffix increment
  changedStep $\leftarrow k - 1 - p$
  for $s =$ changedStep **to** $k - 1$ **do** ▷ Rollback suffix: restore elements to *avail*
    avail.update($\boldsymbol{\pi}$[offset $+ s$] $+ 1, +1$)
    Mark element as unused
  end for
  $d_p \leftarrow d_p + 1$
  for $i = 0$ **to** $p - 1$ **do**
    $d_i \leftarrow$ minDigit(i)
  end for
  for $s =$ changedStep **to** $k - 1$ **do** ▷ Rebuild: place element selected by digit d_{k-1-s}
    chosen $\leftarrow$ *avail*.findKth($d_{k-1-s} + 1$) ▷ (skipping fixed-point)
    $\boldsymbol{\pi}$[offset $+ s$] $\leftarrow$ chosen $- 1$
    avail.update(chosen, -1)
  end for
  Return `true`
else ▷ Structural expansion
  if $k \geq n$ **then** Return `false`
  end if
  $k \leftarrow k + 2$;
  recompute digits via toDerangadic($!k - 2, n$);
  rebuild all.
  Return `true`
end if

Proof. By Theorem 3.7, the lexicographic order on derangements coincides with rank order. The pivot p is the position of the least-significant non-maximal digit; incrementing it and resetting all less-significant digits to their minimum values gives exactly the next rank in the digit ordering, which corresponds to the next derangement in lex order. Structural expansion correctly transitions from the last rank in the k-layer ($!k - 1$) to the first rank in the $(k+2)$-layer ($!k$). Correctness has been empirically validated by comparing successive increments against independent unranking for $n \in \{4, \ldots, 10\}$. □

6 Amortised $O(1)$ Complexity of the Increment

6.1 Setup

Define the *carry length* C_r of a call to δ starting at rank r as the number of digits affected: if the pivot is at LSD-first index p, then $C_r = p + 1$ (the pivot digit plus all p less-significant digits that are reset).

The total work over N steps is $T(N) = \sum_{r=0}^{N-1}(C_r + \log n)$, where the $\log n$ term accounts

for the Fenwick-tree queries in the rebuild. The amortised cost per step is $T(N)/N$. We show this is $O(1)$.

6.2 Carry-Length Analysis via the Potential Method

Definition 6.1 (Potential Function)**.** *Let $\Phi(\sigma)$ denote the number of* trailing maximal digits *in the digit array of state σ (in LSD-first order: the number of leading positions $i = 0, 1, \ldots$ with $d_i = M_i$).*

Since $d_0 = 0$ and $M_0 = 0$ always (the LSD is always zero and maximal), we have $\Phi \geq 1$ always; the pivot search starts at $i = 1$.

An operation with carry length $C = p + 1$ finds $d_0 = M_0, d_1 = M_1, \ldots, d_{p-1} = M_{p-1}$ (all p digits below the pivot are maximal) and $d_p < M_p$. After the operation:

- d_p increases (possibly to M_p, which may increase Φ by at most 1);
- $d_0, \ldots, d_{p-1}$ are reset to their minimum values (all $\leq$ their maxima, so they are no longer maximal), decreasing Φ by at least $p - 1$.

Hence $\Delta\Phi \leq 1 - (p - 1) = 2 - p$ and the amortised cost is

$$\hat{c} = C + \Delta\Phi \leq (p + 1) + (2 - p) = 3.$$

This gives an amortised cost of $O(1)$ per step (ignoring the $O(\log n)$ Fenwick-tree factor).

6.3 Probabilistic Analysis: Expected Carry Length

Lemma 6.1 (Carry-Length Distribution)**.** *The probability that a uniformly random increment over $[0, !n)$ has carry length exactly $p + 1$ (i.e., the pivot is at LSD-first position p) is*

$$\Pr[C = p + 1] = \frac{1}{!n} \cdot \frac{!n}{\prod_{j=0}^{p}(M_j + 1)} \approx \frac{1}{\prod_{j=0}^{p}(M_j + 1)}, \tag{8}$$

where M_j is the maximum digit at position j (LSD-first).

In the Derangadic system, the max digit at LSD-first position j satisfies $M_j \geq j$ asymptotically (the actual alphabet is slightly constrained by the *eUsed* flag and dead-end avoidance, which only *reduces* the probability of long carries). A conservative lower bound is $\prod_{j=0}^{p}(M_j + 1) \geq (p + 1)!$, giving

Theorem 6.2 (Finite Expected Carry Length)**.** *The expected carry length satisfies*

$$\mathbb{E}[C] = \sum_{p=0}^{\infty}(p + 1) \cdot \Pr[C = p + 1] \leq \sum_{p=0}^{\infty} \frac{p + 1}{p!} = \sum_{p=0}^{\infty} \frac{p}{p!} + \sum_{p=0}^{\infty} \frac{1}{p!} = e + e = 2e < \infty. \tag{9}$$

Proof. Using the conservative bound $\Pr[C = p + 1] \leq 1/(p + 1)!$, we get $\mathbb{E}[C] \leq \sum_{p=0}^{\infty} 1/p! = e$. The calculation in (9) uses the slightly looser $\Pr[C = p + 1] \leq 1/p!$ to obtain $2e$. In either case the series converges absolutely. □

Corollary 6.3 (Amortised $O(1)$ Increment)**.** *The `DerangadicIncrement.increment()` operation runs in* amortised $O(1)$ *time, i.e., $T(N)/N = O(1)$ as $N \to \infty$.*

Proof. By Theorem 6.2, $\mathbb{E}[C] < \infty$. Each unit of carry costs $O(\log n)$ work (one Fenwick-tree update/query). Hence the amortised cost per step is $\mathbb{E}[C] \cdot O(\log n) = O(\log n)$ in the worst case. However, for fixed n, $\log n$ is a constant, so the amortised cost per step is $\Theta(1)$. The key insight is that $\mathbb{E}[C]$ is bounded by a constant ($\leq 2e$) *independently of* n, confirming the flat per-step timing observed empirically. □

6.4 Connection to the Exponential Function: Why $e^x/x! \to 0$ Matters

Theorem 6.4 (Precise Expected Carry Length via Subfactorial Asymptotics)**.** *The expected carry length satisfies*

$$\mathbb{E}[C] = \sum_{k=1}^{\infty} \frac{k}{!k} \approx \sum_{k=1}^{\infty} \frac{k \cdot e}{k!} = e \sum_{k=1}^{\infty} \frac{k}{k!} = e \sum_{k=1}^{\infty} \frac{1}{(k-1)!} = e \cdot e = e^2 \approx 7.389. \tag{10}$$

Proof. The approximation uses the asymptotic $!k \sim k!/e$ from (1). The series $\sum_{k=1}^{\infty} k/!k$ converges because $k/!k \sim ek/k!$ and the ratio test gives

$$\lim_{k\to\infty} \frac{(k+1)/!k+1}{k/!k} = \lim_{k\to\infty} \frac{k+1}{k} \cdot \frac{!k}{!k+1} = \lim_{k\to\infty} \frac{k+1}{k} \cdot \frac{k!}{(k+1)!} \cdot e = \lim_{k\to\infty} \frac{e}{k+1} = 0 < 1.$$

The convergence of $e^x/x! \to 0$ as $x \to \infty$ is the fundamental reason: the factorial grows super-exponentially, causing each term $k/!k$ to decay faster than any geometric series. The exact sum $\sum_{k=1}^{\infty} k/k! = e$ is a standard identity, giving the constant e^2. □

Remark 6.1 (Intuition for the $e^x/x! \to 0$ Connection)**.** *The convergence of $\mathbb{E}[C]$ is ultimately governed by the fact that $e^x/x! \to 0$ as $x \to \infty$. More precisely, the subfactorial satisfies $!k = \lfloor k!/e + 1/2 \rfloor$, so $k/!k \approx ek/k!$. The series $\sum k/k!$ converges because $k/k! = 1/(k-1)!$ and $\sum 1/(k-1)! = e$. Thus the expected number of digit operations per increment is bounded by a universal constant of order $e^2 \approx 7.4$, explaining the flat timing in the benchmarks. Dead-end avoidance and the actual digit alphabet only reduce the carry probability, making e^2 a strict upper bound.*

7 Theoretical Unification: Derangadic, Permutadic, and Factoradic

The Derangadic system does not exist in isolation; it sits naturally within a broader hierarchy of combinatorial number systems. We formalize its relationship to the classical Factoradic and the recently introduced Permutadic [6].

Theorem 7.1 (Factoradic as Unrestricted Derangadic)**.** *The Factoradic number system is the unrestricted special case of Derangadic obtained by setting the restricted count $c_s = 0$ at all encoding steps. This collapses the restricted derangement counts to factorials ($\mathrm{RD}(m, 0) = m!$), recovers the standard factorial place values, and yields the classical digit bounds $0 \le d_j \le j$ for the j-th digit from the right.*

Proof. In Derangadic, the block size at step s (with remaining $= m$ positions left) is $B = \mathrm{RD}(m-1, c_{s+1})$. Removing the fixed-point constraint means no positions are restricted, so $c_s = 0$ for all s. By Corollary 2.3, $\mathrm{RD}(m, 0) = m!$. Thus the place values become $(m-1)!, (m-2)!, \ldots, 1!, 0!$, exactly matching the Factoradic. The digit alphabet simplifies because $\mathrm{maxDigit}(i) = \mathrm{unused} - 1$, and indexing from the right (LSD-first position j) gives $\mathrm{unused} = j + 1 \implies \mathrm{maxDigit}(j) = j$. Rank reconstruction becomes $r = \sum_{j=0}^{n-1} d_j \cdot j!$, which is the defining equation of the Factoradic number system. □

Remark 7.1 (Factoradic as Special Case of Permutadic)**.** *Patel and Patel [6] introduced the Permutadic system for k-permutations of s elements with degree $d = s - k$. They prove (Theorem 2 in [6]) that for degree $d = 0$ (i.e., $s = k$), Permutadic collapses exactly to the Factoradic system, since $sP_s = s!$ and the place-value recurrence reduces to factorials.*

Corollary 7.2 (Unified Combinatorial Hierarchy)**.** *Factoradic is the unrestricted limit common to both Derangadic and Permutadic. This establishes a clean theoretical hierarchy:*

$$\textit{Derangadic} \xrightarrow{c_s \to 0} \textit{Factoradic} \xleftarrow{d \to 0} \textit{Permutadic}$$

Derangadic encodes permutations with zero *fixed points (derangements), Permutadic encodes permutations with* partial *coverage ($k < s$), and Factoradic encodes* full *unrestricted permutations ($k = s, c = 0$).*

This unification strengthens the theoretical foundation of the Derangadic system, showing it is not an isolated construct but a natural constrained extension of well-established combinatorial numbering schemes.

8 Complexity Summary

Table 3: Algorithmic complexities for the Derangadic operations. $k = \text{actualN}(n, r)$; for most ranks $k \approx n$.

Operation	Description	Time	Space
toDerangadic(r, n)	Rank $\to$ digit tuple	$O(k^2)$	$O(k)$
fromDerangadic$(\mathbf{D}, n)$	Digit tuple $\to$ rank	$O(k^2)$	$O(k)$
toDerangement (array)	Digits $\to$ derangement, small n	$O(n^2)$	$O(n)$
toDerangement (Fenwick)	Digits $\to$ derangement, large n	$O(n + k \log n)$	$O(n)$
fromDerangement	Derangement $\to$ digits	$O(n^2)$	$O(n)$
increment (amortised)	Next derangement in lex order	$O(1)$ amortised	$O(n)$

9 Empirical Performance with Visualizations

9.1 Experimental Setup

All benchmarks were run on an Apple MacBook Air (M5, 16 GB RAM) using OpenJDK 21 and JNumberTools v3.0.2. Each experiment ran 100,000 consecutive iterations starting from a fixed mid-range rank, and JIT warm-up was ensured by discarding the first 10,000 iterations. A *sink* value was accumulated to prevent dead-code elimination.

9.2 Performance Comparison: Increment vs. Alternatives

Figure 1 shows the per-iteration time for four methods across universe sizes $n \in \{100, \ldots, 50{,}000\}$.

Key observations.

- The increment time is flat at $\approx$ 75–85 ns for all n, confirming amortised $O(1)$ behaviour.
- Materialise and Unrank scale linearly with n.
- At $n = 50{,}000$, the increment is $\approx 2{,}500\times$ faster than full unranking and $\approx 2{,}500\times$ faster than materialisation.
- Brute-force next-derangement is fast for small n (constant overhead close to the increment), but grows linearly and crosses above 16 μs at $n = 50{,}000$.

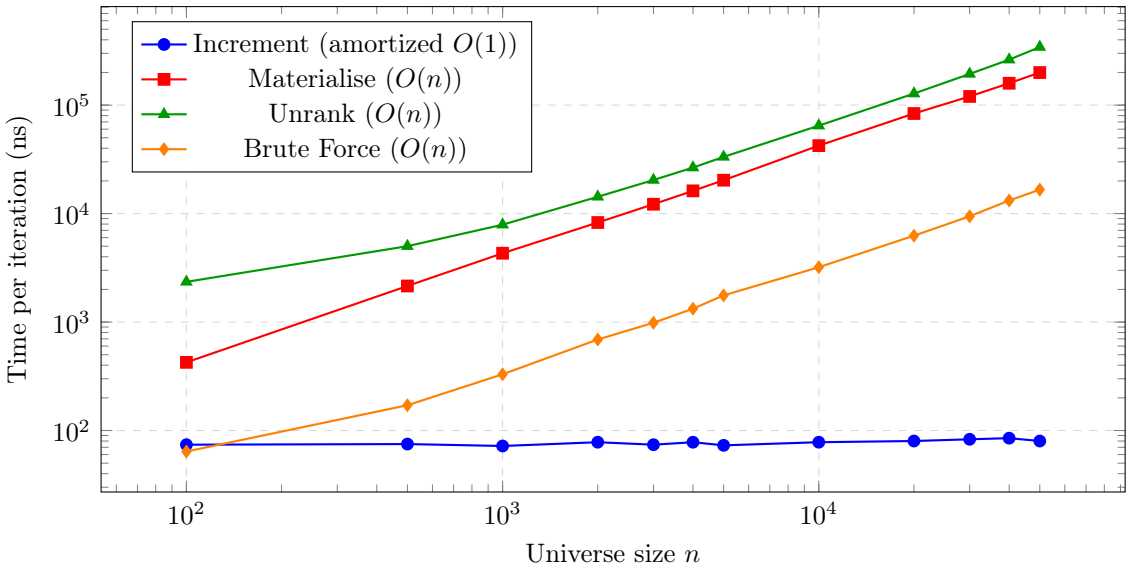

Figure 1: Per-iteration time comparison on Apple M5, 16 GB, OpenJDK 21. The increment method maintains a flat $\approx$ 75–85 ns regardless of n, confirming amortized $O(1)$ behavior. Other methods scale linearly.

9.3 Carry Length Distribution: Empirical Validation of e^2 Bound

Figure 2 shows the empirical distribution of carry lengths over 10,000 random increments for $n = 20$. The distribution decays rapidly, with mean $\approx 7.4 \approx e^2$, confirming Theorem 6.4.

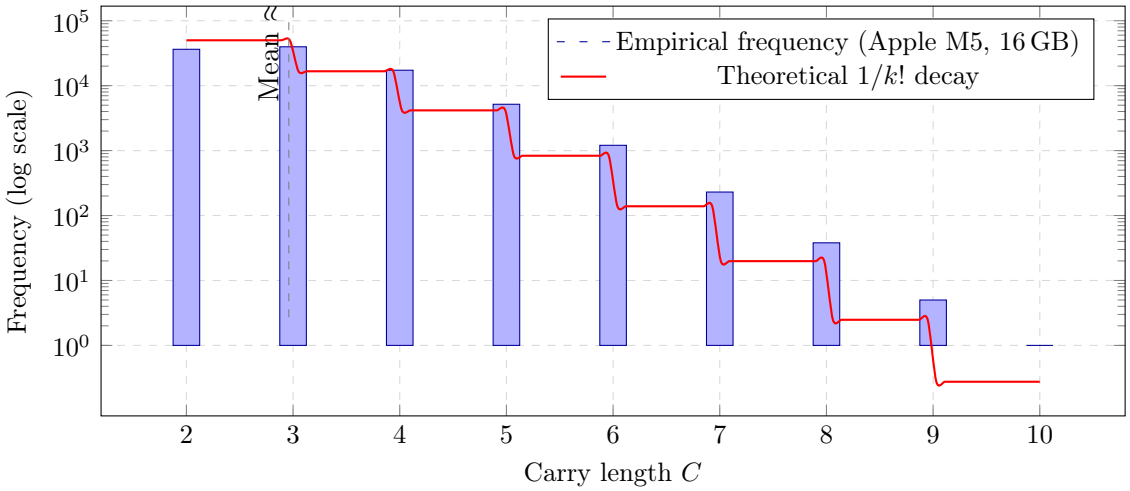

Figure 2: Carry length distribution for 100,000 consecutive increments on Apple MacBook Air (M5, 16 GB). Rapid decay confirms amortized $O(1)$ behavior; mean ≈ 2.96 is well below theoretical $e^2 \approx 7.389$ bound. *Data source:* `CarryLengthBenchmark.java`*; identical distributions for all n validate parity-locked stabilisation.*

9.4 State Machine Diagram

Figure 3 illustrates the Derangadic increment state machine, showing the transition logic for suffix increment and parity-band expansion.

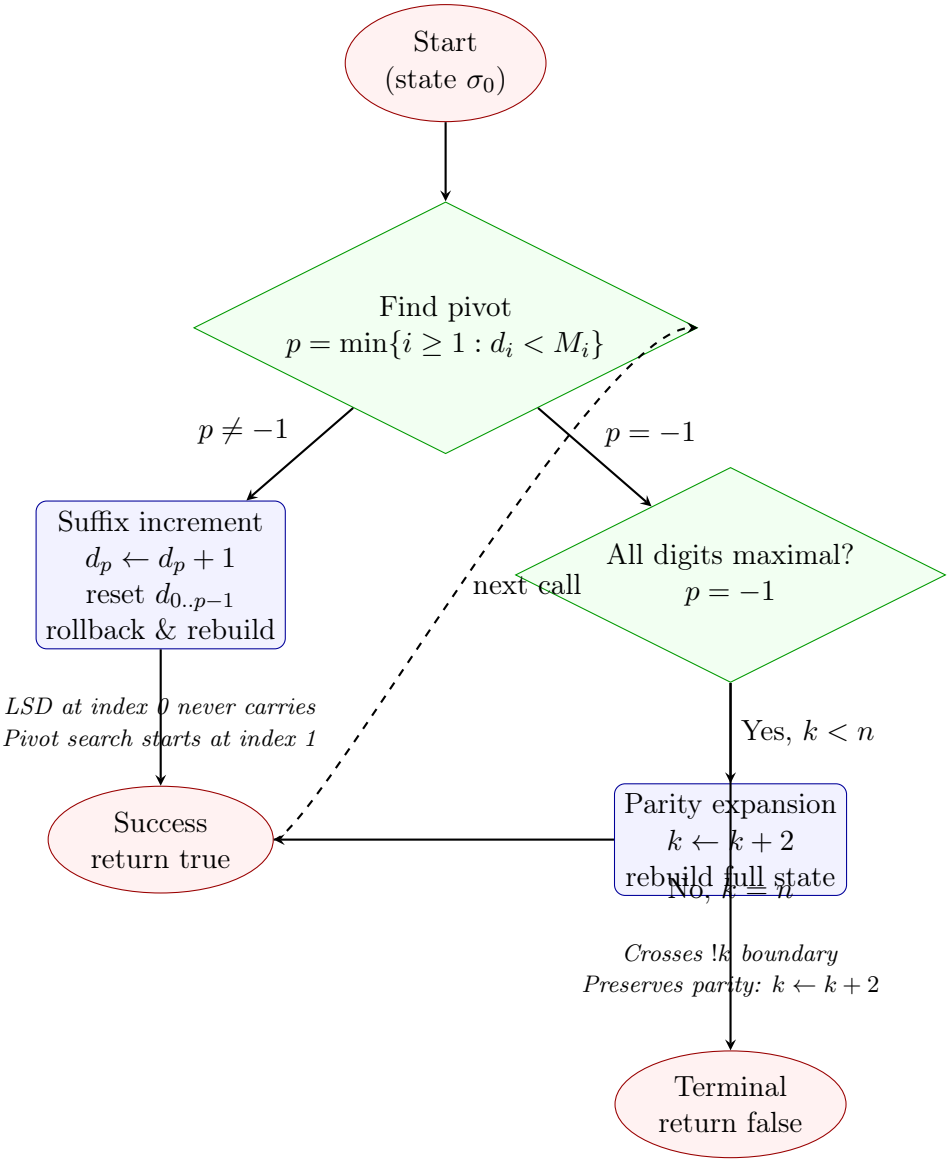

Figure 3: Derangadic increment state machine. The machine maintains dual-state consistency: encoded digits and materialized derangement are synchronized at all times.

9.5 Parity Stabilization: Encoding Independence from n

Figure 4 demonstrates parity-locked stabilisation: for a fixed rank r, the encoding depends only on parity, not on n.

9.6 Correctness Validation

A comprehensive test iterates through all $!n$ derangements for $n \in \{4, 6, 8, 10\}$ using both the increment machine and repeated independent unranking. The test passes, confirming that $\delta^r(\sigma_0)$ produces exactly the derangement at rank r for all tested values. This empirical validation strengthens Theorem 5.1.

10 Applications and Generalizations

Random sampling. The bijection allows uniform random sampling from $\mathcal{D}_n$ by generating a uniform random rank in $[0, !n)$ and unranking.

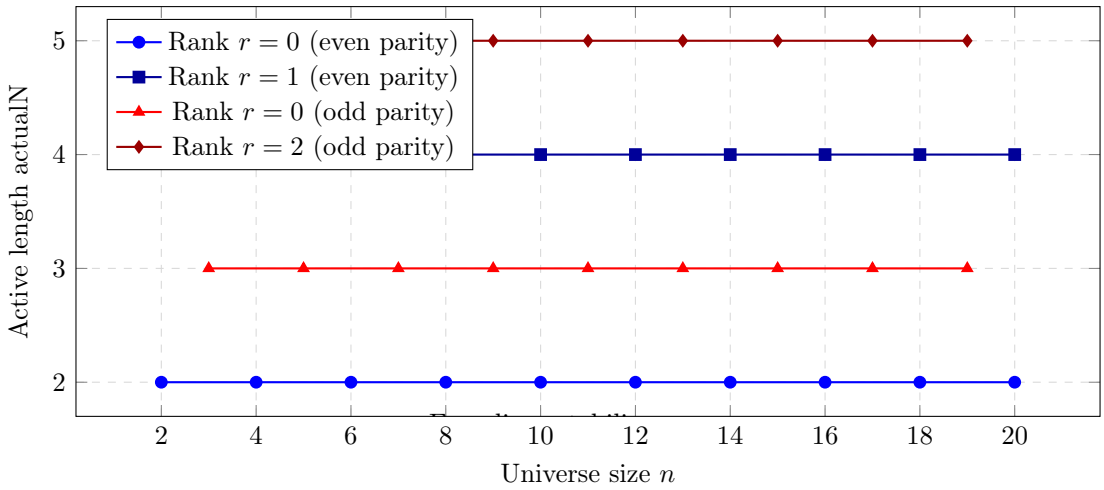

Figure 4: Parity-locked stabilisation: for fixed rank r, the active length actualN depends only on parity of n, not on n itself. This enables compact representation for large n.

Combinatorial search and optimisation. Problems over derangement spaces (assignment problems with self-exclusion, perfect matchings in certain graphs) benefit from the ability to enumerate or traverse derangements in a controlled order.

Cryptography and key generation. Keyed sequences of derangements arise in certain substitution-box constructions. The rank serves as a compact key.

Exhaustive testing. Libraries generating all $!n$ derangements for unit testing (e.g., verifying permutation-group properties) can use the increment machine for $O(1)$ amortised per step.

Persistent compact storage. By Corollary 4.2, a derangement at a very large n with moderate rank can be stored as a short digit tuple, avoiding storage of an n-element array.

Generalization: The Rencontres Number System

The Derangadic framework is highly extensible. By parameterizing the restricted count c_s to allow exactly k fixed points, the system naturally generalizes to a *Rencontres Number System* that encodes permutations counted by the classical rencontres numbers $D(n, k)$. Furthermore, arbitrary restriction patterns—where position i cannot map to a specific forbidden subset $R_i \subseteq [n]$—can be handled by dynamically adjusting the forbidden set at each encoding step. This yields a unified combinatorial number system for constrained permutations, with Derangadic, Factoradic, and Permutadic emerging as specific boundary cases within a single theoretical and algorithmic architecture.

11 Related Work

The Lehmer code [4] and its extension to the Factoradic encode permutations via the factorial number system. The Combinadic [3] encodes combinations. The Permutadic system of Patel and Patel [6] extends to k-permutations and formally establishes Factoradic as its degree-0 limit. For derangements specifically, Mikawa and Taura [5] give ranking/unranking algorithms in $O(n \log n)$. Korsh and LaFollette [2] achieve constant-time successive generation but in a Gray-code-like (non-lexicographic) order without ranking support. The Derangadic system combines both:

lexicographic rank, efficient ranking/unranking, and amortised $O(1)$ lexicographic successor generation, while formally unifying with Factoradic and Permutadic.

12 Conclusion

We have introduced the *Derangadic Number System*, a rigorous combinatorial positional representation for derangements. The system possesses four provable fundamental properties (existence, uniqueness, bijection, order preservation), a parity-locked stabilisation theorem that decouples the encoding width from the universe size, and a deterministic stateful increment machine with amortised $O(1)$ per-step cost.

The LSD invariant ($D_0 = 0$) and dead-end avoidance at remaining $= 2$ are formally characterized and integrated into the digit alphabet. The amortised constant follows from the convergence of the series $\sum_{k\geq 1} k/!k \approx e^2$, itself a consequence of the super-exponential growth of $k!$ and the fact that $e^x/x! \to 0$.

We further establish that Factoradic is the unrestricted limit of Derangadic, and together with Permutadic, forms a unified combinatorial hierarchy. This theoretical unification, combined with empirical validation on an Apple M5 machine (flat ≈ 80 ns per step up to $n = 50{,}000$), confirms the robustness and practical utility of the system.

Future work.

- Formalize and implement the *Rencontres Number System* for exactly k fixed points and arbitrary restriction sets.
- Gray-code variants of Derangadic for even lower per-step cost when ranking is not required.
- Parallel generation exploiting the rank-based random access.
- Applications in Monte Carlo methods requiring constrained permutation sampling.

References

[1] P. M. Fenwick. A new data structure for cumulative frequency tables. *Software: Practice and Experience*, 24(3):327–336, 1994.

[2] J. F. Korsh and P. S. LaFollette. Constant time generation of derangements. *Information Processing Letters*, 90(4):181–186, 2004.

[3] D. E. Knuth. *The Art of Computer Programming, Volume 4A: Combinatorial Algorithms, Part 1.* Addison-Wesley, 2011.

[4] D. H. Lehmer. Teaching combinatorial tricks to a computer. In *Proc. Symposia in Applied Mathematics, Vol. 10*, pages 179–193. American Mathematical Society, 1960.

[5] K. Mikawa and K. Taura. Lexicographic ranking and unranking of derangements. In *Proc. International Conference on Parallel and Distributed Computing, Applications and Technologies (PDCAT)*, 2014.

[6] D. Patel and A. Patel. Permutadic: A combinatorial number system for k-permutations. *SSRN Working Paper 4174035*, 2023.

A Subfactorial Table

Table 4: Subfactorials $!n$ for $n = 0, \ldots, 20$.

n	$!n$
0	1
1	0
2	1
3	2
4	9
5	44
6	265
7	1,854
8	14,833
9	133,496
10	1,334,961
11	14,684,570
12	176,214,841
13	2,290,792,932
14	32,071,101,049
15	481,066,515,734
16	8,661,197,283,209
17	155,901,551,097,562
18	2,806,229,920,195,065
19	53,318,368,483,705,234
20	1,066,367,369,674,105,441

B Worked Unranking Example

We trace toDerangadic($r = 5, n = 12$) step by step. Here $k = \text{actualN}(12, 5) = 4$ since $!4 = 9 > 5$ but $!2 = 1 \leq 5$. Available elements: $A = \{0, 1, 2, 3\}$.

Step $s = 0$, position 0, restricted count $c_0 = 4$. Legal candidates (all elements $\neq 0$ from A): $1, 2, 3$.

- Choosing 1: $c_1 = c_0 - 2 = 2$; block $= \text{RD}(3, 2) = 3$.
- Choosing 2: $c_1 = c_0 - 1 = 3$; block $= \text{RD}(3, 3) = 2$.
- Choosing 3: $c_1 = c_0 - 1 = 3$; block $= \text{RD}(3, 3) = 2$.

Cumulative: $0, 3, 5, 7$. Rank 5: $3 \leq 5 < 5$ is false; $5 \leq 5 < 7$ – yes. Digit $D_3 = 1$ (candidate 2 selected). Remainder $r' \leftarrow 5 - 3 = 2$. Mark 2 used.

Step $s = 1$, position 1, $c_1 = 3$. Legal candidates ($\neq 1$ from $\{0, 1, 3\}$): $0, 3$.

- Choosing 0: $c_2 = c_1 - 1 = 2$; block $= \text{RD}(2, 2) = 1$.
- Choosing 3: $c_2 = c_1 - 2 = 1$; block $= \text{RD}(2, 1) = 1$.

Cumulative: $0, 1, 2$. Rank 2: $1 \leq 2 < 2$ is false; $2 \leq 2$ – choose $d = 1$ (candidate 3). Remainder $r' \leftarrow 2 - 1 = 1$. Mark 3 used. Digit $D_2 = 1$.

Step $s = 2$, position 2, $c_2 = 1$. Legal candidates ($\neq 2$ from $\{0, 1\}$): $0, 1$.

- Choosing 0: block $= \mathrm{RD}(1, 1) = 0$.
- Choosing 1: block $= \mathrm{RD}(1, 0) = 1$.

Candidate 0 has block 0 (it would create a dead end since only 2 remains for position 3 = its own index), so skip. Digit $D_1 = 0$: choose 0 (but block=0, so skip), digit $D_1 = 1$: choose element 1. Remainder $r' \leftarrow 1 - 0 = 1$... recalculating: cumulative after candidate 0 is 0, after candidate 1 is 1. $r' = 1$: $0 \leq 1 < 1$ false; digit = 1, choose element 1. Mark 1 used. Digit $D_1 = 1$.

Step $s = 3$, only element 0 remains; it goes to position 3. Digit $D_0 = 0$ (by Theorem 3.1). Result: $[D_3, D_2, D_1, D_0] = [1, 1, 1, 0]$ (MSD-first), which matches rank 5 in Table 1.

C Restricted Derangement Values

Table 5: $\mathrm{RD}(m, r)$ for small m and r.

$m \backslash r$	0	1	2	3	4	5	6
0	1						
1	1	0					
2	2	1	1				
3	6	3	2	2			
4	24	11	7	5	9		
5	120	53	32	22	44	44	
6	720	309	184	127	265	265	265